Lecture 39: Predicting the Wrong Thing

Joint-embedding predictive architectures

Every number in this deck was checked against its paper on 8 August 2026.

Same model. Same masking. One thing changed.

Take one architecture, one masking scheme, one dataset. Train it twice. The only difference: what it is asked to predict.

- **Run A** predicts the **pixels** of the hidden region.
- **Run B** predicts a **representation** of the hidden region.

Run A is given **60% more training** than run B.

Same model. Same masking. One thing changed.

Take one architecture, one masking scheme, one dataset. Train it twice. The only difference: what it is asked to predict.

- **Run A** predicts the **pixels** of the hidden region.
- **Run B** predicts a **representation** of the hidden region.

Run A is given **60% more training** than run B.

Linear evaluation on 1% of ImageNet:

pixels 40.7 vs **representations 66.9**

A gap of **26.2 points**, with the losing run getting more compute.

(Assran et al., 2023, Table 7.)

The question this chapter asks

Fifteen chapters have quietly been answering one question
without ever asking it out loud:

What should a model be asked to predict?

Not *which architecture*. Not *which optimiser*. The **objective**.

Four answers you have already seen

Chapter	Predicts	Target comes from
ch8 autoencoders	the input itself	the data
ch10 diffusion	a distribution	the data
LLMs	the next token	the data
ch12 CLIP	*whether* two things match	the data
ch16 JEPA	**a representation**	**a network being trained**

Every target above is fixed before training starts. The last one is not: it is produced by a second encoder that is **learning at the same time**.

That single change is the whole chapter.

Outline

Three ways to build a self-supervised task

You have already seen two of them.

The joint-embedding architecture, via CLIP

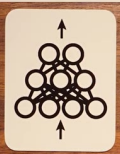
Chapter 12, renamed. Two encoders, one shared space, agreement as the objective.

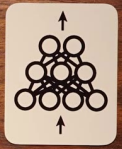
- Take two views of the same thing: two crops, an image and its caption, two signatures.
- Encode both. Train so matching pairs land close together.

The idea is older than CLIP by three decades. LeCun and colleagues built it at Bell Labs in the early 1990s to detect **fraudulent signatures**: pass a reference and a candidate through two copies of the same network, and compare the vectors that come out. They called them **Siamese networks**.

Nothing is generated. The network never draws a signature.

[0.2, -0.1, ... , 0.3]





Welch Labs (2026)

What a joint-embedding architecture cannot do

It answers *"do these two things match?"* and nothing else.

> Give it the left half of a photograph and the right half. It can score them as compatible. It **cannot tell you what the right half looks like** - not in pixels, not in features, not at all. There is no mechanism in the architecture for expressing *how* y relates to x.

For a world model that is fatal. *"Is this a plausible next second of video?"* is useful. *"What is the next second of video?"* is what an agent actually needs.

The generative architecture

Hide part of the input. Reconstruct it. Score the reconstruction against the truth.

This is the shape of the denoising autoencoder (ch8), of masked autoencoders, and - with the axis relabelled from space to time - of next-token prediction.

It fixes the joint-embedding problem exactly: it says *what y* is. And it cannot collapse, because the target is fixed by the data and the model does not get a vote.

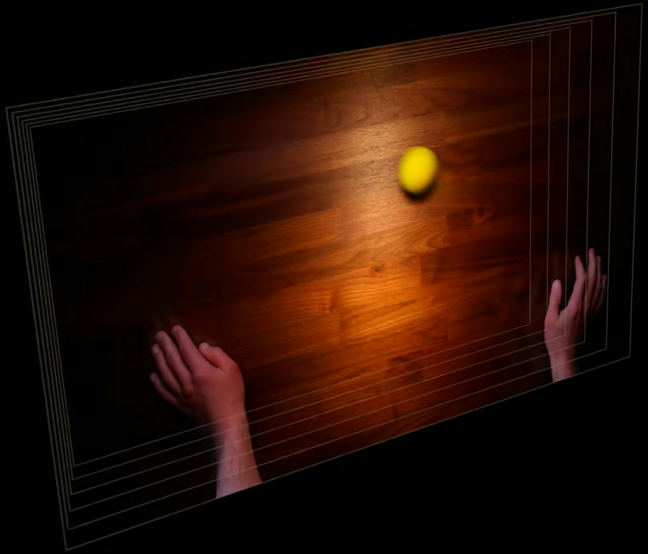
The generative architecture

Hide part of the input. Reconstruct it. Score the reconstruction against the truth.

This is the shape of the denoising autoencoder (ch8), of masked autoencoders, and - with the axis relabelled from space to time - of next-token prediction.

It fixes the joint-embedding problem exactly: it says *what y* is. And it cannot collapse, because the target is fixed by the data and the model does not get a vote.

> So why is this not the end of the lecture? Because for video and images it produces **blur**, and the reason it produces blur is not a bug to be engineered away.



Welch Labs (2026)

Why: you cannot enumerate the futures

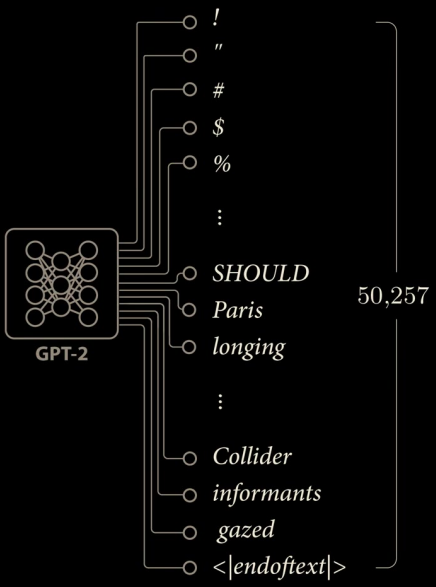
A language model has a **finite vocabulary**. GPT-2 has exactly **50,257** possible next tokens, one output per token, and it can raise or lower each independently.

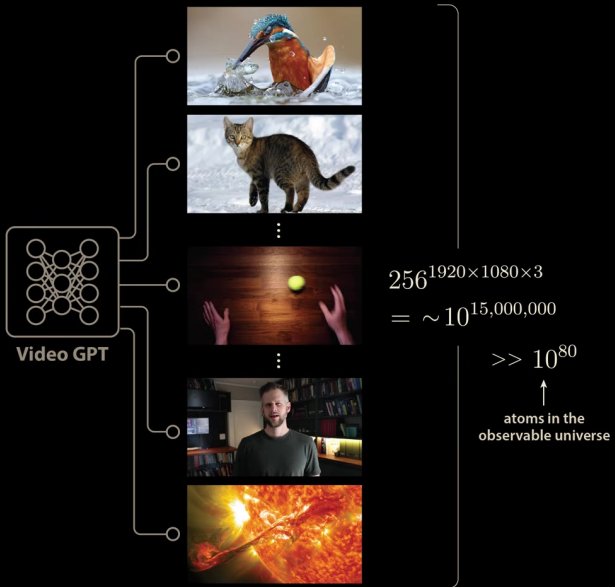
Video has no such luxury. Each subpixel of a full-HD frame takes one of 256 values, and there are $1920 \times 1080 \times 3$ of them:

$$256^{1920 \times 1080 \times 3} \approx 10^{15{,}000{,}000} \text{ possible next frames}$$

against roughly 10^{80} atoms in the observable universe.

There is no output layer for that. So the network must emit **pixel values directly** - and now uncertainty has nowhere to go.





Welch Labs (2026)

So it predicts the average, and the average is a blur

A ball rolls toward the edge of a table. In the training data it sometimes bounces left, sometimes right.

Language model. Separate outputs for *"left"* and *"right"*. It raises **both**. The distribution is the answer.

Video model. One frame out. Squared error is minimised by the **conditional mean** - so it draws the ball in **both places, faintly**.

Third time this course has met this argument.

Chapter 15: averaging two valid robot trajectories gives a third, invalid one. Chapter 10: diffusion exists partly to avoid committing to a mean. Same disease every time - **a single-valued prediction of a multi-valued thing**.

The ball bounced to the *left*

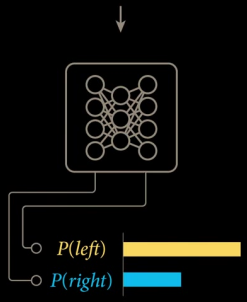
The ball bounced to the *right*

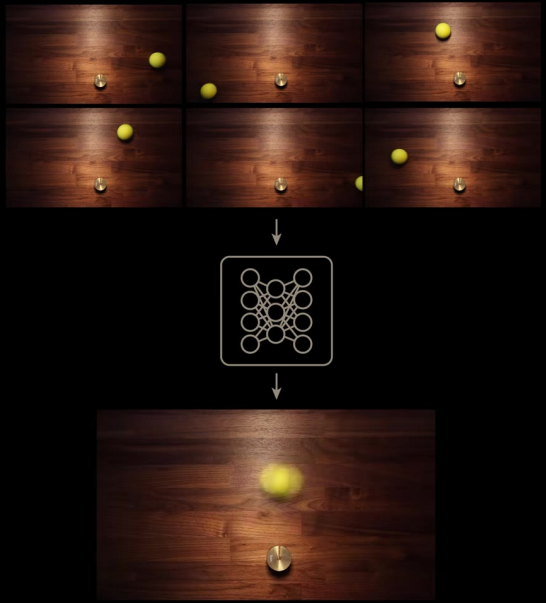
The ball bounced to the *right*

The ball bounced to the *left*

The ball bounced to the *left*

The ball bounced to the *left*





And most of those pixels never mattered

Blur is the visible symptom. The deeper cost is **where the capacity goes**.

> A model trained to predict a dashcam feed spends most of its resources predicting **the motion of the leaves** on the trees beside the road. Those leaves are essentially unpredictable - and they are a very large number of pixels.

The loss does not know the difference between *"a car is pulling out"* and *"that leaf moved 3 pixels left"*. It charges the same price for both, and there are far more leaves than cars.

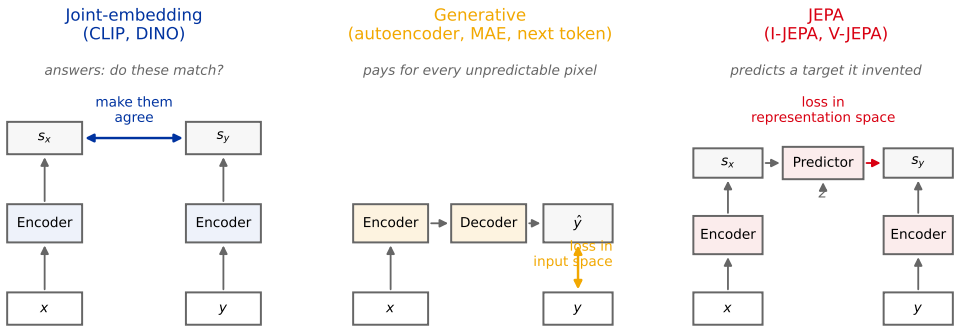
This example is LeCun's. On the next slide, the right-hand panel is the **optical flow** of that road scene - colour shows which direction each pixel is moving. Almost all the motion in the frame is foliage.

Source: Can Driver

Welch Labs (2026)

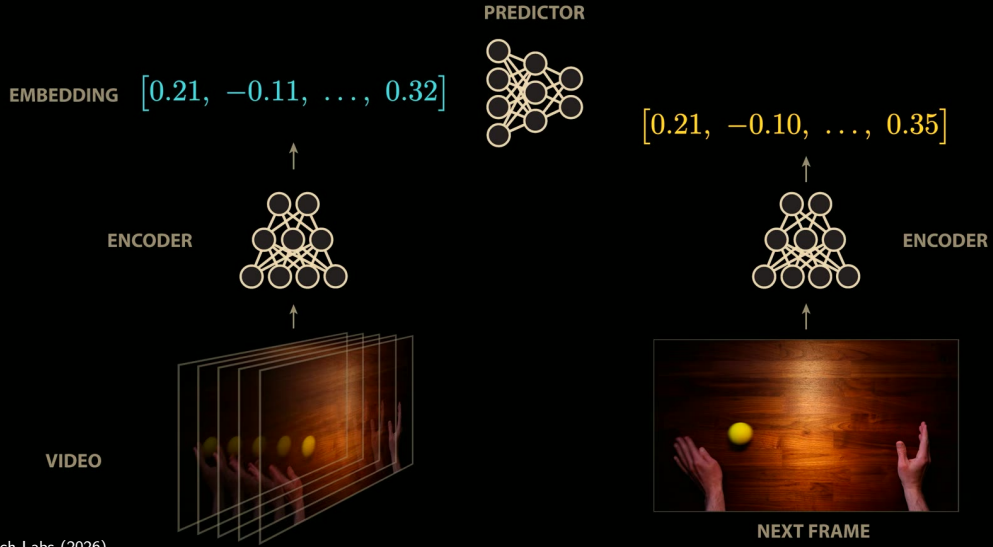
JEPA: predict the representation, not the thing

Encode *both* sides. Then predict one embedding from the other, through a **predictor** network.



The latent z absorbs what is genuinely unpredictable - which way the ball bounced. Two things are new: the loss lives in **representation space**, and the target is produced by a network that is **itself being trained**.

JOINT EMBEDDING PREDICTIVE ARCHITECTURE (JEPA)



Welch Labs (2026)

The sentence to remember

JEPA gives the model permission to throw information away.

If the target encoder may discard the leaves, the loss stops charging for them.

Careful: none of that was I-JEPA

Every example so far - the bouncing ball, the dashcam - has been **video**. That was to show why *predicting pixels* is hopeless in general.

> The first working JEPA, **I-JEPA**, has **no time axis at all**. It takes **one still photograph**, hides some regions, and predicts their representations from the rest of the same image. Nothing is a "next frame".

Predictive here means *predicts one part from another part*, not *predicts the future*. Time arrives in L40, and only then does any of this become a world model.

The catch: collapse

And the rest of the design exists to stop it throwing *everything* away.

Write down the degenerate solution

Let the target encoder map **every** input to the same constant vector c. Let the predictor ignore its input and output c.

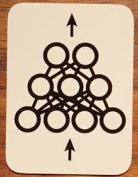
$$\| \text{Pred}(s_x) - s_y \| \;=\; \|c - c\| \;=\; \mathbf{0}$$

The loss is **exactly zero** - the best value it can possibly take. The representation is worthless: every image, every caption, every video maps to the same point.

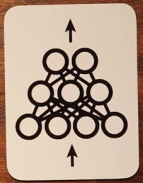
This is **representation collapse**, and nothing in the objective forbids it.

The generative architecture cannot do this, because it is scored against pixels it does not control. That safety is exactly what we gave up.

[-0.4, 0.2, ... , -0.4]



[-0.4, 0.2, ... , -0.4]



This is not mode collapse

Same word, different failure. Chapter 8b used it for something else.

Mode collapse (ch8b, GANs)
A *generator* covers one mode of the data. It produces real-looking output, but always the same kind.

Representation collapse (here)
An *encoder* maps every input to one point. It produces nothing at all, and the loss is delighted.

One is a generator being lazy about *variety*.
The other is an encoder being lazy about *information*.

The loss will not tell you it broke

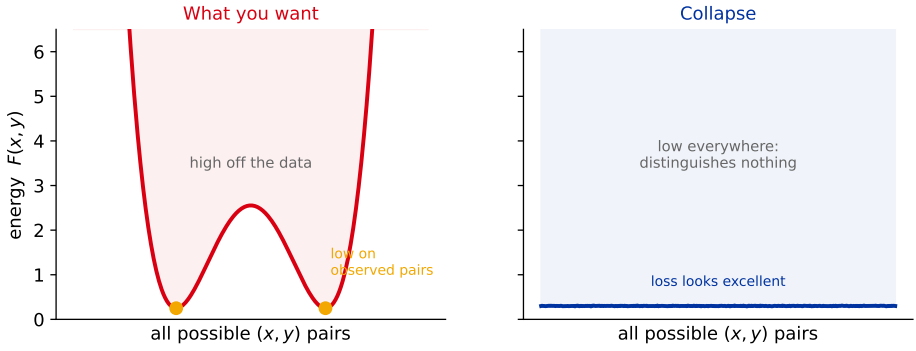
A falling loss curve is the thing you normally trust. Here it proves nothing, for **three** separate reasons.

1. **Zero is reachable by cheating.** The degenerate solution attains the global minimum. A low loss is consistent with a useless model.
2. **The target is moving.** The loss measures agreement with the target encoder, which is itself being updated. It is not comparable across runs, or even across time within one run.
3. **Shrinking beats learning.** The loss is a distance between embeddings, so **scaling the target encoder's outputs toward zero reduces it** without changing anything the representation encodes.

You cannot early-stop on this loss. Model selection needs a **downstream probe** - which is expensive, so far fewer configurations get tried than you are used to.

The energy picture

Define an energy $F(x, y)$ that is low when x and y are compatible. Training pushes it down on observed pairs.



The failure mode is **not** low energy on the data - that is the goal. It is low energy **everywhere**: a flat landscape that distinguishes nothing. So the real objective is always *low on the data, and kept high off it*.

One loss, three choices

Every JEPA objective in the literature has the same two-term shape:

$$\mathcal{L} \;=\; \underbrace{\mathbb{E}\big[\, d\big(\mathrm{Pred}(s_x, z),\, s_y\big)\big]}_{\text{latent predictive invariance}} \;+\; \lambda \cdot \underbrace{R(s_1, \ldots, s_B)}_{\text{anti-collapse}}$$

Reading it: s_x, s_y are the two embeddings; d is a distance; z is the optional latent that absorbs what is genuinely unpredictable (which way the ball bounced); $s_1, \ldots, s_B$ are the embeddings of a **batch** of B examples; and λ is a single weight trading the two terms off.

Everyone agrees on the first term. **The families differ only in what R is** - and in one case, in whether it is a term in the loss at all rather than a property of the architecture.

The energy $F(x, y)$ from the previous slide is the same quantity as that first term, viewed one pair at a time.

The three families

Family	What R is	Examples	Cost
Contrastive	not a regulariser - a second data term: pull matching pairs together, and explicitly push *non*-matching pairs apart	**CLIP** (ch12)	needs many negative pairs; scales badly with dimension
Teacher–student	*absent from the loss.* Collapse is blocked **structurally**, by making the two branches asymmetric	**I-JEPA**, DINO	works, poorly understood; silent failure mode
Moment-matching	an explicit penalty on batch statistics: keep every dimension's variance up, decorrelate the dimensions	**Barlow Twins**	one more term and weight to tune

I-JEPA is in the middle row - worth dwelling on, because it is the row with no theory behind it.

One representative each; BYOL, SimCLR, MoCo, VICReg and SIGReg belong to the same three rows and are not separate ideas.

The teacher that resists being dragged

I-JEPA's answer is an **exponential moving average** (**EMA**) teacher. The target encoder is never touched by gradient descent. It is a slow copy of the context encoder:

$$\theta_{\text{target}} \leftarrow m \cdot \theta_{\text{target}} + (1 - m) \cdot \theta_{\text{context}} \qquad m : 0.996 \rightarrow 1.0$$

Two supporting details, both load-bearing:

- **Stop-gradient.** No gradient flows into the target branch. The student chases the teacher; the teacher never chases back.
- By the end of pretraining $m \rightarrow 1$, so the teacher **freezes** entirely.

Why this blocks collapse: to reach the constant solution the student would have to *drag the teacher there too* - but the teacher only moves at rate $1 - m \approx 0.004$ per step, and it is the thing defining the target. The constant is no longer a fixed point you can reach in one step.

Say it plainly: a heuristic, not a theorem

Keep the intuition from the last slide - it is genuinely why this works in practice. But be precise about what it does *not* establish.

It argues the constant solution is **hard to reach quickly**. It does **not** prove the constant solution is unreachable, nor that some *partial* collapse - a representation using only a few of its dimensions - is prevented at all.

There is **no general proof** that an EMA teacher prevents collapse. It is an empirical recipe inherited from BYOL and DINO, and it is contested.

- **C-JEPA** argues I-JEPA's EMA *does not fully prevent collapse*, and bolts VICReg's variance and covariance terms on to the same backbone to fix it.
- **SALT** (2025) reports that a **frozen** teacher beats an EMA teacher at matched compute.
- **LeJEPA** (2025) removes the teacher, the stop-gradient and the predictor altogether.

The load-bearing mechanism of the most cited paper in this area is a trick nobody has justified.

The other teacher-student method: DINO

DINO shares I-JEPA's row, and it is the one that wins the benchmarks at the end of this lecture - so it deserves its mechanism stated, not just its name.

- Same asymmetry: a **student** network and an **EMA teacher**, with a stop-gradient on the teacher.
- But the two branches see **different hand-crafted crops** of the image - a global view for the teacher, small local views for the student - and the student must match the teacher's output distribution across views.
- Anti-collapse comes from two extra tricks on the teacher's outputs: **centring** (subtract a running mean, stopping one class dominating) and **sharpening** (a low temperature, stopping the output going uniform).

The difference that matters here: **DINO needs you to hand-design the augmentations.** I-JEPA needs you to hand-design the **masking** instead. Neither is free. Remember that when we get to the scoreboard.

The family that came back

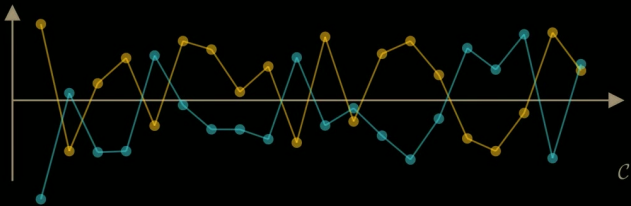
The third row has the best origin story, and it is where the field may be heading.

LeCun's own account: the joint-embedding methods were *"kind of hacks"* until **Stephane Deny** brought in **Horace Barlow's 1961** hypothesis - that neurons in a visual system work by **reducing redundancy between each other**.

Barlow Twins applies it literally:

1. Take the batch of activations from each of the two encoders.
2. Compute the **cross-correlation matrix** between them.
3. Drive it toward the **identity**: diagonal $\rightarrow 1$ (the two views agree), off-diagonal $\rightarrow 0$ (the dimensions stop duplicating each other).

VICReg is the simplified successor; **SIGReg** (LeJEPA, 2025) the current one. Keep this row in mind - the last frame of this deck comes back to it.

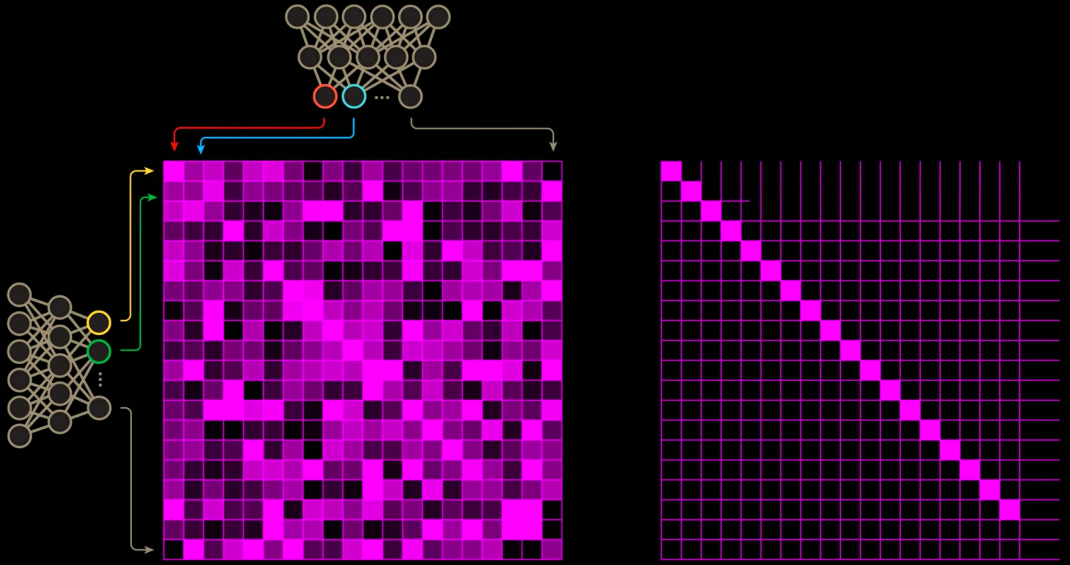


-0.77

$$\mathcal{C} \triangleq \frac{\sum_b z_b^A z_b^B}{\sqrt{\sum_b \left(z_b^A\right)^2}\sqrt{\sum_b \left(z_b^B\right)^2}}$$

PEARSON CORRELATION COEFFICIENT

Welch Labs (2026)



Welch Labs (2026)

I-JEPA, in detail

The first one that worked.

Three networks, and the one you keep

All three are Vision Transformers (ch12).

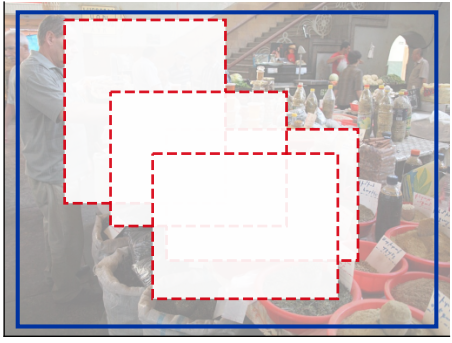
Context encoder	standard ViT. Sees **only the visible patches**. Trained by gradient descent.
Target encoder	same architecture. **Not** trained by gradients - EMA of the context encoder.
Predictor	a deliberately **narrow** ViT: width fixed at **384** whatever the backbone, depth 6 / 12 / 16.

After pretraining you **keep the target encoder** and **throw the predictor away**. The predictor existed only to make the task solvable.

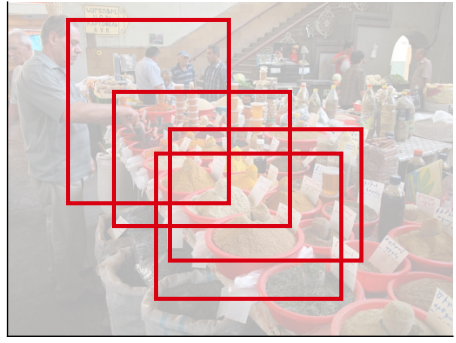
Remember that. Next lecture reverses it exactly.

Multi-block masking

Context: one large block, minus any overlap
scale (0.85, 1.0), about 25% of patches survive



4 targets: scale (0.15, 0.2), aspect (0.75, 1.5)
large enough to be semantic, not texture



Sample **4 target blocks** at scale $(0.15, 0.2)$ and aspect ratio $(0.75, 1.5)$. Sample **one context block** at scale $(0.85, 1.0)$, then **delete from it any region overlapping a target** - otherwise the task is trivial. About **25%** of patches survive as context.

The subtle bit: targets are masked at the *output*

The target blocks are selected **after** the target encoder has run, not before.

> The target encoder sees the **entire, unmasked image**. Only then do we pick out the patch embeddings belonging to the target blocks.

Why it matters: a patch embedding computed *in context* already reflects the whole scene, so it is a **semantic** target. Encode the block in isolation and you are back to predicting local texture.

Be honest about the cost: the two branches now see different amounts of evidence. The JEPA loss is not the likelihood of anything.

How we are going to read the numbers

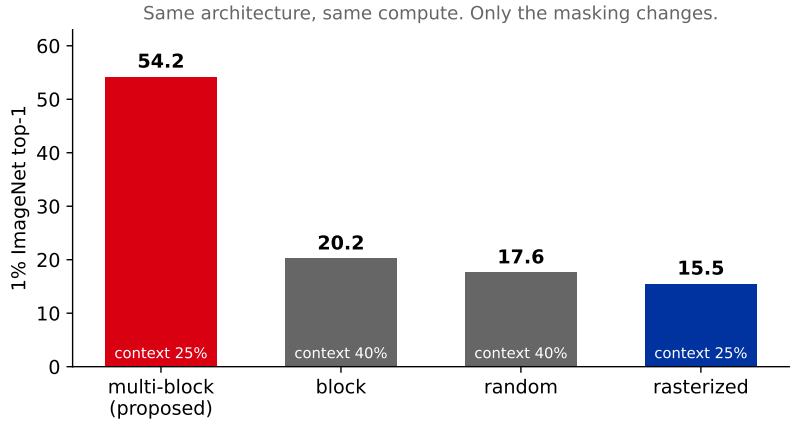
Every result in this deck uses a **linear probe** on a **frozen** encoder.

1. Pretrain with no labels. Then **freeze the encoder completely**.
2. Train **one linear layer** on top of it, with labels, to classify.
3. Report that accuracy.

A linear layer can only *read off* what is already there. It cannot fix a bad representation. So this measures **how much usable structure the pretraining put in** - not how good the whole system can be made.

It is also the honest choice: fine-tuning the whole network would let a good optimiser paper over a weak representation.

Ablation 1: the masking strategy is not a detail



A **34 to 39 point** spread from how you cut up the image, at fixed architecture and fixed compute. Larger than most architectural choices this course has taught.

Read the rasterized row

Rasterized uses one image quadrant as context: **25% of the patches**, exactly the same budget as the winning strategy.

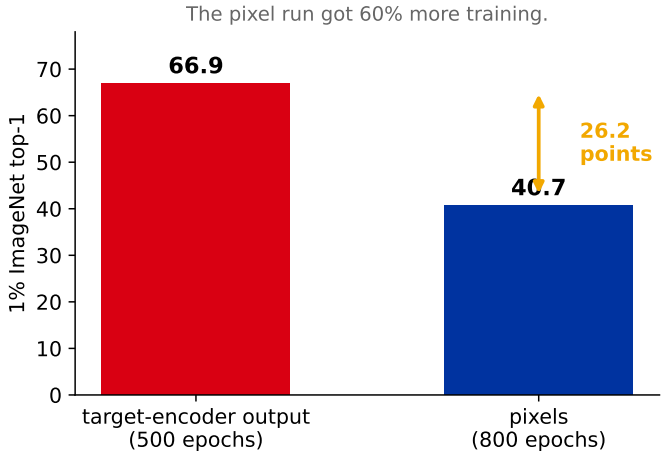
It scores **15.5** against **54.2**.

Same amount of information by the patch count. **39 points worse** by where the information sits.

Two rules come out of this:

- Targets must be **large enough to be semantic**. A small target is a texture patch, and predicting texture teaches you texture.
- Context must be **spatially distributed**. A single corner of an image tells you almost nothing about the rest of it.

Ablation 2: the target space



The frame-one result. Same architecture, same masking; only the target changes. If this chapter has one number, it is this one: **what you predict matters more than what you predict it with.**

What the predictor learns to represent

Train a separate decoder purely to *look* at what the predictor outputs (it is not part of I-JEPA). Across samples of the same prediction:

Consistent: the pose and the object part. The back of a bird. The roof of a car. It knows *what* is there.

Varies wildly: exact texture, background, fine detail. It has **declined to represent** them.

That is the architecture working as designed: representing what is predictable, and staying **silent** about what is not.

Caveat honestly: hand-picked visualisations in a paper, using a decoder the method does not otherwise have.

What I-JEPA still gets wrong: it assumes it knows *where*

I-JEPA's predictor is told the **exact position** of the patch it must predict, via fixed positional embeddings.

Given only part of a dog, **you cannot locate its tail precisely.**
The position is genuinely uncertain - and the model is not allowed to say so.

StoP-JEPA models each masked position as a **Gaussian random variable** with learned covariance instead of a fixed point. A few lines of code, no extra compute, better probing.

This is the bouncing-ball argument again - applied to **position** instead of content. Fourth appearance. It keeps being the same lesson: *do not force a single-valued answer onto a multi-valued question.*

Does it work?

This is the part most summaries get wrong.

ImageNet-1k, linear evaluation

No hand-crafted augmentations

Method	Arch	Top-1
MAE	ViT-H/14 (1600 ep)	77.2
data2vec	ViT-L/16 (1600 ep)	77.3
CAE	ViT-L/16 (1600 ep)	78.1
I-JEPA	**ViT-H/14 (300 ep)**	**79.3**
I-JEPA	**ViT-H/16_{448} (300 ep)**	**81.1**

With augmentations

Method	Arch	Top-1
SimCLR v2	RN152 (2×)	79.1
DINO	ViT-B/8 (300 ep)	80.1
iBOT	ViT-L/16 (250 ep)	81.0

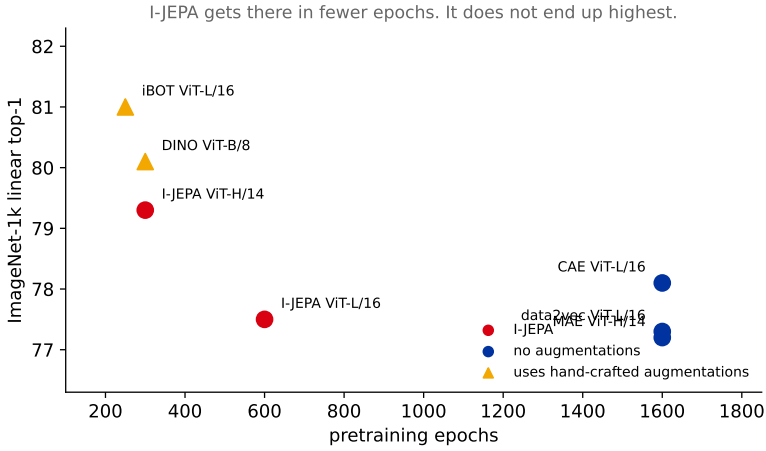
Read it carefully before you repeat it.

I-JEPA **clearly wins its own category**, at a quarter to a half the epochs.

It does **not** beat DINO or iBOT at 224px. Only the 448px variant matches them.

The paper's own words: it *"decreases the gap"*.

What it does win: efficiency



A ViT-Huge/14 trained on **16 A100s in under 72 hours** - less compute than a ViT-*Small*/16 trained with iBOT. The reason is structural: augmentation-based methods process **ten or more crops** of every image per step. I-JEPA processes **one view**, and its context encoder sees only a quarter of that.

A slide about where numbers come from

A widely-circulated summary of this paper reports:

"I-JEPA achieves **72.4%** *on 1% ImageNet against MAE's* **59.8%**."

The paper's own Table 2 says:

I-JEPA ViT-H/14, 300 epochs: **73.3**
MAE ViT-H/14, 1600 epochs: **71.5**

A claimed 12.6-point gap is really **1.8 points**. Inflated roughly sixfold.

And the true story is the better one: I-JEPA got there in **300 epochs against 1600**. The real result is efficiency, not a blowout.

Where the image line actually went

Uncomfortable epilogue. The default general-purpose image encoder in 2026 is **DINOv2/v3** - self-distillation **with** hand-crafted augmentations, the family I-JEPA was positioned against.

> DINOv3 (August 2025) reached **88.4** on ImageNet - against a supervised state of the art of 88.6. Its authors: *"the first time that a self-supervised model has reached comparable results to weakly and supervised models on image classification."*

Not comparable to the 79.3 above - different protocol, different era, vastly more data. The point is not the number. The point is that Meta ships both, and the one people build on is not the JEPA.

Being right about the objective did not automatically win the benchmark.

Does that sink the thesis? No - but be precise about what survives. The *target-space* result (66.9 vs 40.7) is a controlled comparison and stands. What has *not* been shown is that **dropping augmentations** is also a win: DINOv3 keeps them and leads. Two separate claims, and this chapter only established the first.

DINOv3

Oriane Siméoni* Huy V. Vo* Maximilian Seitzer* Federico Baldassarre* Maxime Oquab*
Cijo Jose Vasil Khalidov Marc Szafraniec Seungeun Yi Michaël Ramamonjisoa
Francisco Massa Daniel Haziza Luca Wehrstedt Jianyuan Wang
Timothée Darcet Théo Moutakanni Leonel Sentana Claire Roberts
Andrea Vedaldi Jamie Tolan John Brandt[1] Camille Couprie
Julien Mairal[2] Hervé Jégou Patrick Labatut Piotr Bojanowski

Meta AI Research [1]*WRI* [2]*Inria*

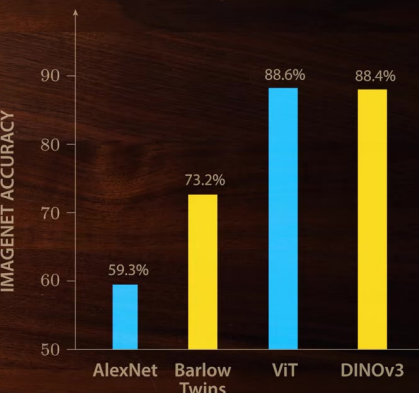
**corresponding authors: {osimeoni,huyvo,seitzer,baldassarre,qas}@meta.com*

Abstract

Self-supervised learning holds the promise of eliminating the need for manual data annotation, enabling models to scale effortlessly to massive datasets and larger architectures. By not being tailored to specific tasks or domains, this training paradigm has the potential to learn visual representations from diverse sources, ranging from natural to aerial images—using a single algorithm. This technical report introduces DINOv3, a major milestone toward realizing this vision by leveraging simple yet effective strategies. First, we leverage the benefit of scaling both dataset and model size by careful data preparation, design, and optimization. Second, we introduce a new method called Gram anchoring, which effectively addresses the known yet unsolved issue of dense feature maps degrading during long training schedules. Finally, we apply post-hoc strategies that further enhance our models' flexibility with respect to resolution, model size, and alignment with text. As a result, we present a versatile vision foundation model that outperforms the specialized state of the art across a broad range of settings, without fine-tuning. DINOv3 produces high-quality dense features that achieve outstanding performance on various vision tasks, significantly surpassing previous self- and weakly-supervised foundation models. We also share the DINOv3 suite of vision models, designed to advance the state of the art on a wide spectrum of tasks and data by providing scalable solutions for diverse resource constraints and deployment scenarios.

1 Introduction

Foundation models have become a central building block in modern computer vision, enabling broad generalization across tasks and domains through a single, reusable model. Self-supervised learning (SSL) is a powerful approach for training such models, by learning directly from raw pixel data and leveraging the natural co-occurrences of patterns in images. Unlike weakly and fully supervised pretraining methods (Radford et al., 2021; Dehghani et al., 2023; Bolya et al., 2025) which require images paired with high-quality metadata, SSL unlocks training on massive, raw image collections. This is particularly effective for training large-scale visual encoders thanks to the availability of virtually unlimited training data. DINOv2 (Oquab et al., 2024) exemplifies these strengths, achieving impressive results in image understanding tasks (Wang et al., 2025) and enabling pre-training for complex domains such as histopathology (Chen et al., 2024). Models trained with SSL exhibit additional desirable properties: they are robust to input distribution shifts, provide strong global and local features, and generate rich embeddings that facilitate physical scene understanding. Since SSL models are not trained for any specific downstream task, they produce versatile and robust generalist features. For instance, DINOv2 models deliver strong performance across diverse tasks and domains without requiring task-specific finetuning, allowing a single frozen backbone to serve multiple purposes. Importantly, self-supervised learning is especially suitable to train on the vast amount of available observational data in



"All in all, this is the first time that a self-supervised model has reached comparable results to weakly and supervised models on image classification"

Where this leaves us

One idea that travels, one thing it cannot do, one question still open.

The objective travels

Predicting in representation space is now standard far beyond images:

- **Audio and speech** - A-JEPA, Audio-JEPA, WavJEPA (ch13)
- **Biosignals** - EEG, ECG, brain dynamics, genomics
- **3D and point clouds**, **molecules and polymers**
- **Tabular and time series** - including LaT-PFN, which crosses JEPA with the prior-fitted networks of **ch14**
- **Remote sensing**, **recommendation**, **robotics**

Look at that list again. Almost every one is a domain with **few labels and abundant unlabelled data** - which is exactly the regime where "pretrain without labels or augmentations" pays.

What it structurally cannot do

A question you should be asking after ch10: *why not just use JEPA instead of diffusion?*

JEPA is **not designed to reconstruct anything**. It deliberately discards the information a decoder would need. No pixels, no waveforms, no tokens.

They are not applying for the same job. Diffusion generates; JEPA understands.

The reason it is good at understanding is the reason it cannot generate.

This will matter in L40: a world model you cannot look inside is harder to debug than one that draws you a picture.

What is still unresolved

The middle row of the taxonomy - the one I-JEPA depends on - is under active attack.

- **SALT** (Sept 2025) reports a **frozen** teacher beating EMA at matched compute, and that student quality is remarkably insensitive to teacher quality.
- **LeJEPA** (Nov 2025) proves the **isotropic Gaussian** is the optimal embedding distribution and enforces it with a single regulariser (SIGReg) - deleting the teacher, the stop-gradient and the predictor. Its authors report one hyperparameter and results across 50+ architectures.

The claim that would matter most in practice: **LeJEPA's training loss correlates with downstream accuracy.** If that holds, the expensive-probe problem from earlier in this lecture goes away.

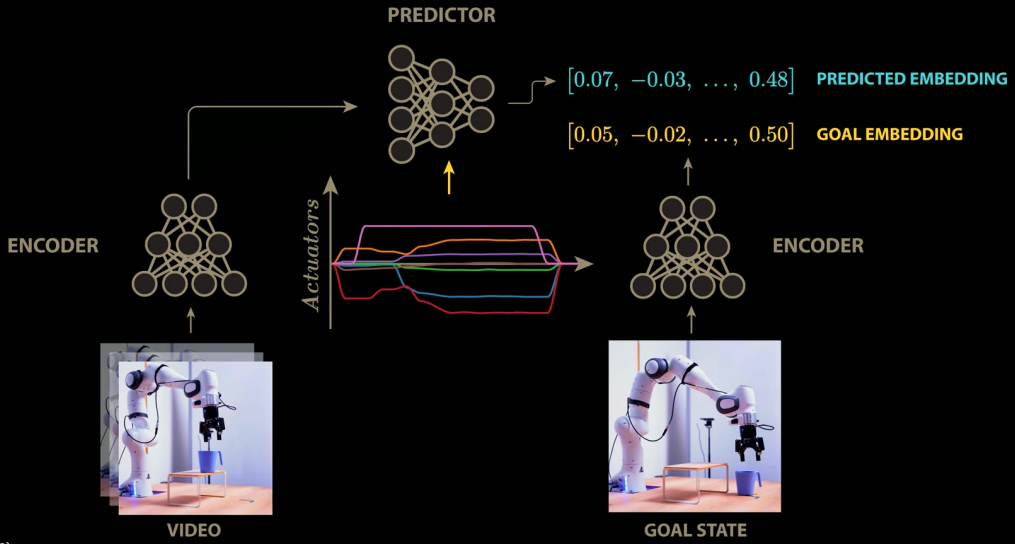
Both are young, and neither has displaced the standard recipe. The moment-matching family may yet win.

Recap

- The choice of **prediction target** dominates. Representation targets beat pixel targets by **26.2 points** with less training.
- Pixel targets fail because they force a **single-valued answer to a multi-valued question**, and because they charge full price for unpredictable detail.
- Learned targets buy that freedom at the cost of **collapse**. All the machinery - EMA teachers, stop-gradients, VICReg, SIGReg - exists to pay it.
- **The loss cannot tell you it worked.** Only a downstream probe can.
- Read scoreboards carefully. I-JEPA wins its category and **does not** beat DINO at equal resolution.

Next: everything today happened inside **one still photograph**. Add a time axis and you get a video model. Add **actions** and you get something you can plan with - a **world model**, and a billion-dollar argument about whether it is the future of AI.

JOINT EMBEDDING PREDICTIVE ARCHITECTURE (JEPA)



Welch Labs (2026)

Note that planning involves autoregressively making predictions, not shown here.